

RESEARCH ARTICLE OPEN ACCESS

Optimization and Benchmarking of Lightweight Neural Networks for Efficient Embedded AI Deployment

Vidapankal Mohammad Fridous¹ | Abhishek Agarwal²  | K. B. Bhaskar³ | V. Nandagopal⁴ | N. Sivakamasundari¹

¹Department of Mechatronics Engineering, Hindustan Institute of Technology and Sciences, Chennai, India | ²Department of Mechanical Engineering, College of Science and Technology, Royal University of Bhutan, Bhutan | ³Department of Electrical and Electronics Engineering, Adhi College of Engineering and Technology, Chennai, India | ⁴School of Engineering, Department of Electrical and Electronics Engineering, Mohan Babu University, Tirupati, Andhra Pradesh, India

Correspondence: Abhishek Agarwal (agarwala.cst@rub.edu.bt)

Received: 23 March 2026 | **Revised:** 11 April 2026 | **Accepted:** 25 April 2026

Keywords: edge computing | embedded AI | hardware-aware benchmarking | lightweight neural networks | model optimization | TinyML

ABSTRACT

The rapid growth of artificial intelligence (AI) has accelerated its integration into embedded systems, enabling real-time intelligence at the edge with reduced latency, improved privacy, and lower power consumption. This paradigm, known as Embedded AI, deploys machine learning and deep learning models directly on resource-constrained platforms such as microcontrollers, FPGAs, and system-on-chip devices. This study presents a structured optimization and benchmarking framework for lightweight neural networks targeting heterogeneous embedded hardware platforms, including CPU, GPU, TPU, and MCU architectures. Model compression techniques such as quantization, pruning, and mixed-precision computation are applied to reduce memory footprint and computational complexity while preserving classification accuracy. Performance evaluation is conducted using inference latency, model size, and power consumption as benchmarking metrics under consistent experimental conditions. Results demonstrate that optimized lightweight models significantly improve computational efficiency and energy performance across edge deployment platforms. The proposed framework provides practical guidance for selecting suitable neural network configurations for real-time embedded AI applications.

1 | Introduction

The convergence of embedded systems and artificial intelligence has significantly changed the design of resource-constrained intelligent computing platforms [1]. Convergence, or Embedded AI, makes local intelligence possible in systems that operate under low-resource environments. Embedded artificial intelligence (AI) systems, enabled through the combination of machine learning (ML) and deep learning (DL) algorithms with microcontrollers, field-programmable gate arrays (FPGAs), and system-on-chip (SoC) platforms, enable real-time

decision-making, adaptive control, and intelligent automation independent of central cloud infrastructure [2].

Traditionally, computations for AI have been offloaded to high-powered cloud servers, which, while offering computational capability, introduce latency, create privacy concerns, and are reliant on ongoing connectivity to the network. Embedded AI, however, demands bringing the computational intelligence closer to the origin of the data at the edge [3]. This kind of strategy greatly improves responsiveness, boosts energy efficiency, and improves data security by reducing transmission. The creation of

This is an open access article under the terms of the [Creative Commons Attribution](https://creativecommons.org/licenses/by/4.0/) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2026 The Author(s). *Engineering Reports* published by John Wiley & Sons Ltd.

AI hardware accelerators like graphics processing units (GPUs), tensor processing units (TPUs), and AI-specialized microcontrollers played a crucial role in the feasibility of running advanced AI models on low-power devices [4, 5].

Among the most important of these is the increasing deployment of embedded GPUs—that is, NVIDIA Jetson modules to provide the parallel compute processing ability needed to execute DL models in real time [6]. The modules leverage technologies such as CUDA programming, tensor cores, and mixed-precision math to enable efficient processing of compute-intensive inference workloads within constrained thermal and power budgets.

Despite all these advancements, some of the downsides of applying AI to embedded systems include low processing power, low memory, energy efficiency, and real-time response [7, 8]. Deep neural networks (DNNs), although being very accurate, are usually bigger than the capacity of embedded hardware. To achieve this, there has been such lightweight neural network architecture as TinyML and MobileNet and optimization strategies like quantization, pruning, and knowledge distillation [9]. Such measures compact the size of the model as well as reduce complexity but with maintained inference accuracy and hence suitable for execution in real time on limited resource devices.

The rapid development of embedded artificial intelligence has enabled practical deployment across multiple application domains requiring real-time and resource-efficient inference. In healthcare systems, embedded AI supports wearable monitoring platforms capable of detecting physiological anomalies locally with minimal latency [10]. In intelligent transportation systems, embedded AI contributes to advanced driver-assistance functionality through on-device perception and decision support mechanisms. Similarly, AI-enabled smart home controllers improve energy management, occupancy awareness, and security monitoring through edge-level inference. In industrial automation environments, embedded AI supports predictive maintenance and quality inspection by enabling continuous equipment monitoring without reliance on cloud connectivity [11]. Across these application scenarios, edge deployment improves responsiveness, reduces communication overhead, and enhances operational reliability under constrained computing conditions.

This study develops a structured optimization and benchmarking framework for lightweight neural networks deployed on heterogeneous embedded platforms including CPU, GPU, TPU, and MCU architectures. The framework integrates quantization, pruning, and mixed-precision computation and evaluates their impact on latency, memory footprint, and energy consumption under comparable experimental conditions. Emphasis is drawn on the GPU as a hard real-time AI workload accelerator on edge hardware. The discussion is extended further to the use of model optimization techniques that render it possible to execute sophisticated AI models within the tight constraints of embedded hardware. Additionally, the research investigates real-world application scenarios, baselines, and future work directions, including newer developments such as neuro-morphic computing and federated learning that will continue to spur the evolution of embedded intelligence.

With the ongoing blurring of the lines between hardware and software via advancements in AI hardware co-design and smart system architecture, Embedded AI is a principal driver of next-generation computing. The framework not only closes the gap between cloud analytics and local real-time decision-making but also sets the stage for green, scalable, and secure intelligent systems across various industries.

Figure 1 presents the overall workflow and organization of the proposed study. The paper first introduces the fundamentals of Embedded AI, followed by the identification of system challenges and lightweight solution strategies. The proposed architecture and optimization techniques are then introduced, after which benchmarking is performed using latency, power, and accuracy metrics. Finally, application scenarios and future research directions are discussed before concluding the study.

This study proposes a unified hardware-aware benchmarking framework for evaluating lightweight neural network optimization strategies across heterogeneous embedded platforms including CPU, GPU, TPU, and MCU architectures under consistent experimental conditions. Unlike existing studies that typically evaluate optimization techniques or deployment performance on individual hardware targets, the present work provides a structured cross-platform comparison integrating quantization, pruning, and mixed-precision computation within a single experimental workflow. The proposed framework enables systematic analysis of latency, power consumption, model size, throughput, and runtime memory utilization, offering practical guidance for selecting efficient lightweight neural network configurations for real-time embedded AI deployment.

2 | Literature Review

2.1 | Embedded Intelligence on FPGA and Edge Devices

Recent studies have investigated the deployment of embedded intelligence on FPGA-based and edge computing platforms to enable low-latency and energy-efficient on-device processing [12, 13]. FPGA implementations support applications such as object detection and surveillance through configurable parallel architectures that improve computational efficiency while maintaining scalability under hardware constraints [14]. Complementary research has further demonstrated that executing machine learning models directly on embedded platforms enhances responsiveness and data security by reducing dependence on centralized cloud infrastructure [15]. These developments highlight the importance of hardware-aware model deployment strategies for achieving reliable real-time inference in resource-constrained embedded environments.

2.2 | AI for Industrial IoT and Healthcare Embedded Applications

Artificial intelligence techniques are increasingly being integrated into embedded Industrial Internet of Things (IIoT) systems to support predictive maintenance, anomaly detection, and real-time monitoring under constrained computational

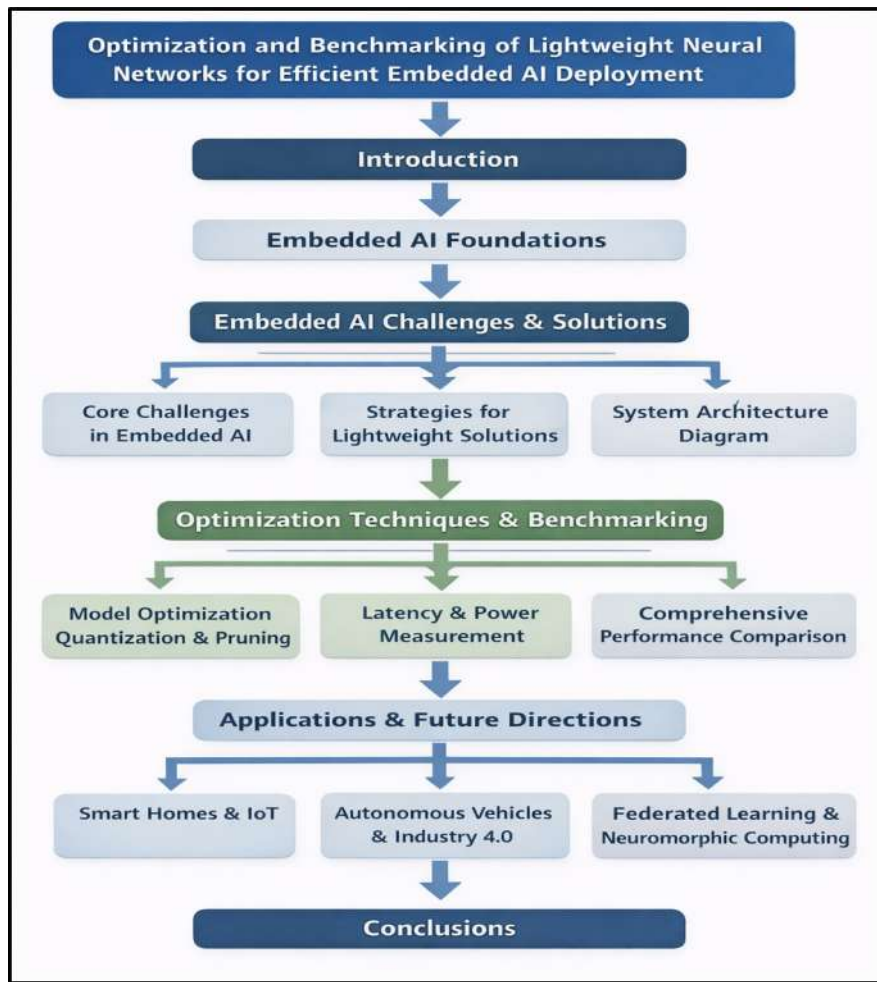


FIGURE 1 | Overall research methodology and organization flow of the proposed embedded AI framework.

environments. Machine learning models such as autoencoders and lightweight neural networks enable efficient feature extraction and fault diagnosis while maintaining low latency and reduced communication overhead in distributed industrial settings [16]. In parallel, embedded machine learning has been widely explored for wearable and ambulatory healthcare systems deployed on microcontroller-based platforms, where energy-efficient inference and hardware-aware model design are essential for continuous physiological monitoring [17]. These application-driven developments demonstrate the growing importance of deploying optimized AI models directly on embedded devices to enable reliable and secure edge intelligence across industrial and healthcare domains.

2.3 | Lightweight Neural Network Architectures for Embedded Systems

Lightweight neural network architectures have emerged as a key enabler for deploying deep learning models on resource-constrained embedded platforms across application domains such as healthcare, automotive systems, and mobile edge devices. Approaches based on compact convolutional neural network architectures and parameter-efficient model design reduce computational complexity while preserving inference accuracy for

real-time decision-making at the edge [18]. These developments emphasize the importance of balancing model performance with memory footprint and latency constraints, thereby supporting efficient implementation of embedded AI solutions in practical deployment environments. Early developments in efficient convolutional neural network architectures such as MobileNetV1 and MobileNetV2 introduced depthwise separable convolutions to significantly reduce computational complexity and memory requirements for mobile and embedded vision applications [19, 20]. Subsequent architectures including EfficientNet further improved parameter efficiency through compound scaling strategies [21, 22]. In addition, structured model compression approaches such as Deep Compression demonstrated the effectiveness of pruning, quantization, and weight sharing for reducing model size while preserving accuracy [23, 24]. Knowledge distillation techniques have also been widely explored for transferring knowledge from large teacher networks to compact student models suitable for deployment on resource-constrained edge platforms [25–27].

2.4 | Hardware Accelerators for Embedded AI Processing

Advancements in machine learning hardware accelerators have significantly improved inference efficiency on embedded

platforms by enabling optimized execution of deep neural networks under strict resource constraints. These accelerators include specialized convolutional neural network processing units and low-power edge AI architectures designed to reduce computational latency and energy consumption while maintaining model performance [28]. Comparative analyses of ultra-compact edge AI accelerators for image recognition further demonstrate the effectiveness of hardware-specific optimization strategies in enabling real-time inference on embedded systems [29]. These developments highlight the growing importance of accelerator-aware deployment frameworks for improving the practicality of embedded AI systems.

2.5 | Model Compression and Optimization Techniques

Model compression and optimization techniques play a critical role in enabling deep neural networks to operate efficiently within resource-constrained embedded systems. Approaches such as quantization, pruning, and structurally efficient neural network design reduce memory footprint, computational complexity, and power consumption while maintaining acceptable inference accuracy across heterogeneous embedded hardware platforms [30]. In addition, recent hardware–software co-design strategies, including tensorized neural network training and low-bitwidth model representation, have demonstrated further improvements in memory utilization, communication bandwidth requirements, and inference latency for edge AI deployment scenarios [31]. These developments highlight the importance of integrated optimization pipelines for supporting scalable and energy-efficient embedded intelligence.

2.6 | Research Gap and Motivation

Although significant progress has been made in embedded AI deployment, lightweight architectures, and hardware acceleration, most existing studies address these aspects independently. A unified framework that integrates lightweight neural network optimization with cross-platform benchmarking of latency, power, and memory efficiency remains limited for heterogeneous cross-platform evaluation involving CPU, GPU, TPU, and MCU architectures within a single experimental workflow. This gap motivates the present work, which proposes an optimized lightweight AI framework and evaluates its performance across diverse embedded hardware platforms. Existing benchmarking initiatives such as MLPerf Tiny primarily focus on standardized microcontroller inference evaluation rather than cross-platform benchmarking across heterogeneous CPU, GPU, TPU, and MCU architectures [32, 33]. Unlike existing studies that evaluate lightweight neural networks on individual hardware platforms, the present work provides a unified cross-platform benchmarking workflow that systematically compares optimization strategies across heterogeneous embedded architectures. A comparative summary of representative studies on embedded AI architectures, lightweight neural network models, hardware accelerators, and optimization strategies is presented in Table 1 to highlight existing contributions and their limitations.

TABLE 1 | Summary of existing works in embedded AI.

References	Focus area	Method/model	Hardware	Key contribution	Limitations
[14]	FPGA embedded AI	Object detection	FPGA	Low latency inference	Limited scalability
[16]	IIoT AI	Autoencoders	MCU/Edge	Predictive maintenance	Dataset specific
[17]	Healthcare TinyML	ML on MCU	Wearables	Ultra-low power	Limited accuracy
[18]	Lightweight CNNs	MobileNet/Tiny models	Edge GPU	Efficient models	Still moderate size
[29]	AI Accelerators	CNN hardware	TPU/ASIC	Fast inference	Costly hardware
[30]	Model compression	Quantization/Pruning	General	Reduced memory and FLOPs	Accuracy trade-off
[31]	Edge AI optimization	Tensorized Neural Networks	Edge Hardware	Ultra-low latency and memory usage	Implementation complexity
[34]	Efficient CNN Models	MobileNetV3	Edge Devices	High Accuracy with reduced parameters	Still computationally demanding
[35]	TinyML Benchmarking	MLprex Tiny	MCU platforms	Benchmarking AI performance on microcontrollers	Limited dataset variety
[36]	TPU Edge Acceleration	Deep Neural Networks	TPU	High Performance inference acceleration	Limited flexibility
[37]	Embedded ML Frameworks	TensorFlow Lite Micro	MCU Systems	Enables ML inference on low-power devices	Limited model complexity
[38]	AI on Mobile platforms	Smartphone Deep Learning	Mobile GPU	AI deployment on consumer devices	High energy consumption

3 | Embedded AI Challenges and Lightweight Solutions

Embedded AI has revolutionized the field of deploying intelligent systems by enabling artificial intelligence capability in edge devices. Embedded AI systems, as compared to cloud-based systems, bring computation and inference nearer to the source of data, enabling low-latency processing, improved data privacy, and self-governing decision-making. These advantages make Embedded AI highly appropriate for use in healthcare, autonomous vehicles, smart manufacturing, and the Internet of Things (IoT). But implementing AI on platforms imposes another set of challenges that require fresh advances in software and hardware to be efficient, reliable, and scalable Figure 2 illustrates the overall architecture of the proposed Embedded AI framework. Lightweight neural network models are first designed and optimized using techniques such as quantization, pruning, and mixed-precision computation. The optimized models are deployed on embedded hardware accelerators including GPUs, FPGAs, and SoC-based platforms. These edge devices enable real-time inference while maintaining low latency, reduced power consumption, and minimal memory usage. The architecture supports diverse applications such as smart homes, autonomous vehicles, industrial IoT, and healthcare systems, while future enhancements include neuromorphic computing and hardware–software co-design for improved efficiency.

Figure 3 illustrates the proposed workflow for efficient embedded AI deployment across heterogeneous hardware platforms. Lightweight neural network architectures such as MobileNetV3 and the custom TinyML model are first optimized using quantization, pruning, and mixed-precision computation to reduce

memory footprint and computational complexity. The optimized models are then converted into deployment-ready formats and executed on CPU, GPU, TPU, and MCU platforms. Finally, performance benchmarking is performed using accuracy, latency, power consumption, model size, throughput, and runtime memory utilization to evaluate deployment suitability under resource-constrained conditions.

The proposed algorithm summarizes the complete optimization and deployment pipeline adopted in this study. It ensures systematic reduction of computational complexity while preserving inference accuracy across embedded hardware platforms (see Algorithm 1).

3.1 | Core Challenges in Embedded AI

A primary limitation of deploying AI models on embedded platforms is restricted processing capability. Standard deep learning models such as convolutional and recurrent neural networks typically require specialized optimization or high-performance computing platforms such as GPUs or cloud-based infrastructure. Such models require top-of-the-line processing, memory bandwidth, and storage unavailable on standard embedded platforms like edge SoCs or microcontrollers.

Memory limitations are the second important concern. Embedded systems only possess a few megabytes or several kilobytes of RAM, and therefore cannot easily fit enormous AI models. Deep neural networks alone take up hundreds of megabytes of memory for inference purposes, not including the memory required for intermediate calculation and system access. In addition, most embedded systems have limited energy budgets in most uses,

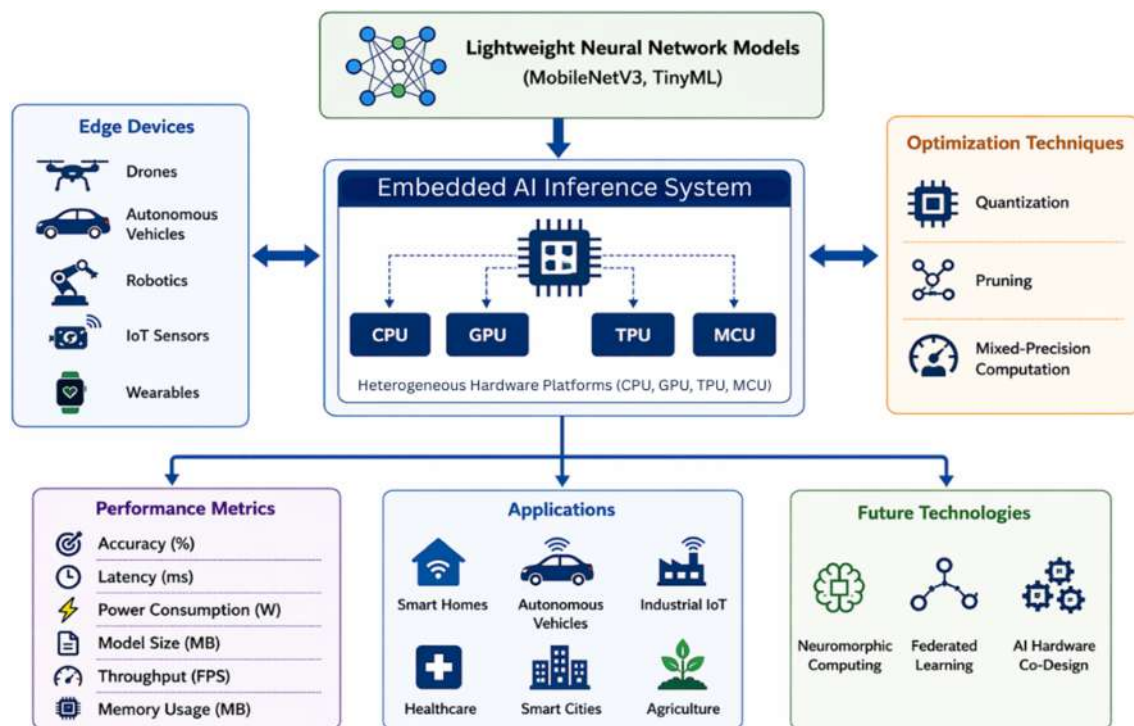


FIGURE 2 | Architecture of the proposed lightweight neural network–based embedded AI inference system across heterogeneous hardware platforms.

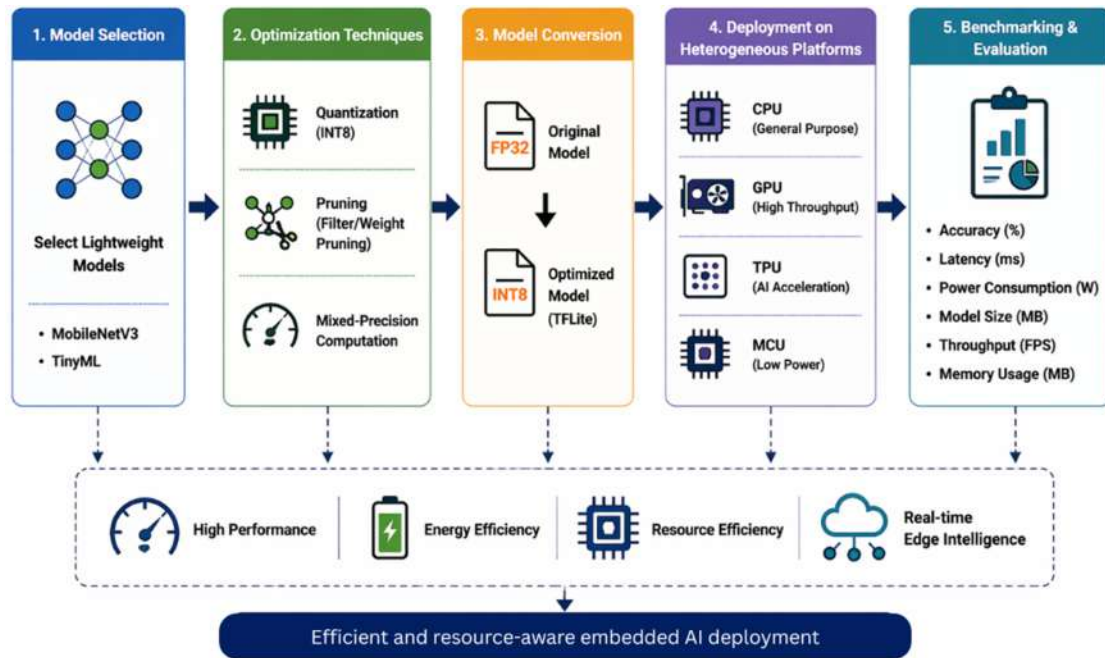


FIGURE 3 | Workflow for efficient embedded AI deployment using lightweight neural network optimization and cross-platform benchmarking.

ALGORITHM 1 | Lightweight Neural Network Optimization and Deployment Framework.

Input: Dataset D , Base model M , Target embedded hardware
Output: Optimized model M_{opt} and performance metrics (latency, power consumption, accuracy, and model size)

- 1: Load dataset D
- 2: Preprocess and split into training and testing sets
- 3: Select lightweight base model M (MobileNetV3/TinyML)
- 4: Train model on training data
- 5: Apply optimization techniques:
 - a) Quantization
 - b) Pruning
 - c) Mixed-precision computation
- 6: Generate optimized model M_{opt}
- 7: Deploy M_{opt} onto target hardware H (CPU/GPU/TPU/MCU)
- 8: Measure performance metrics:
 - Inference latency
 - Model size
 - Power consumption
 - Accuracy
- 9: Compare results with baseline model
- 10: Select best configuration based on accuracy–efficiency trade-off

especially battery-powered or autonomous. General AI computations quite naturally entail vast amounts of power, a feat which is not possible in scenarios where long lifetimes and energy efficiency are the norm.

Real-time processing and heat limits are obstacles to making this a reality. AI models must execute within strict latency constraints to become viable for real-time applications such as autonomous drone obstacle detection or anomaly classification on medical wearables. In addition, thermal constraints of embedded systems

limit sustained high-compute operation over extended periods, requiring careful optimization of model structure and deployment strategy.

Hardware platform heterogeneity further complicates deployment across CPU, GPU, TPU, and MCU architectures. Accelerators (Google Edge TPU or NVIDIA Jetson) to general-purpose microcontrollers (ARM Cortex-M), heterogeneity in architecture makes AI models and frameworks very hardware-aware and adaptable. Heterogeneity makes deployment difficult as the developers require tuning and calibration of AI models on specific target platforms.

3.2 | Strategies for Lightweight Embedded AI Solutions

In connection with these constraints, recent years have seen rapid growth in research on designing lightweight models of AI, as well as optimizing methods for deployment in embedded systems. These research efforts attempt to constrain the amount of resources used without sacrificing the model's capability for producing trustworthy inference.

Lightweight neural network models like MobileNet, SqueezeNet, and TinyML models are specifically designed to minimize the parameters and computations. For instance, MobileNet employs depthwise separable convolutions to deeply reduce the computational expenses while keeping the accuracy level at a competitive rate for image classification and object detection. Similarly, these models are used as the core building blocks for most of the embedded AI applications, with performance vs. efficiency trade-offs.

Complementing these architectural innovations are model compression techniques, including:

- **Quantization:** Reduces numerical accuracy of model parameters, commonly from 32-bit floating point to 8- or even 4-bit integers. In addition to taking up less memory, it also accelerates inference speed, especially on hardware with support for low-precision math.
- **Pruning:** Removes weights or neurons with a minimum contribution to the output. Structured pruning techniques can eliminate whole filters or channels and build sparse networks that are more parallelizable and deployable on embedded devices.

They also include the co-design of hardware and software strategies under which models, as well as features of the platform of interest, are optimized in concert. APIs such as TensorFlow Lite for Microcontrollers and CMSIS-NN, for instance, support the deployment of quantized models at effective throughputs on ARM-based platforms. Likewise, products such as NVIDIA Jetson Nano or Google Coral include toolkits that support hardware-accelerated inference using CUDA or Edge TPU APIs, respectively.

Another avenue of hope is the usage of compiler-level optimizations and inference engines such as TVM, Glow, and ONNX Runtime that convert high-level models into optimized, hardware-knowledgeable binaries. These tools perform automatic operations such as operator fusion, memory reuse, and layout conversion such that the AI models run at maximum efficiency on the target embedded platform.

In short, even though the deployment of AI into embedded systems is a daunting challenge owing to the constraints of computing, memory, and power, an entire slate of architectural and algorithmic approaches now enable AI models to be deployed efficiently into low-resource environments. The complementarity of low-weight neural network design, model optimization strategies, and hardware-optimized implementation is making edge intelligence a reality. In the subsections that follow, this paper will provide examples from real-life, benchmark information, and applications that support the efficacy and effectiveness of these solutions.

3.2.1 | Custom TinyML Architecture Design

In addition to MobileNetV3, a compact custom TinyML convolutional neural network architecture was designed to evaluate performance trade-offs under strict memory and computation constraints typical of microcontroller-based inference platforms. The architecture consists of three convolutional layers with kernel sizes of 3×3 , each followed by batch normalization and ReLU activation, and two max-pooling layers for spatial feature reduction. The network accepts input images resized to 224×224 pixels to maintain compatibility with the MobileNetV3 baseline architecture and enable consistent benchmarking across platforms. The convolutional feature extractor is followed by a fully connected classification layer with softmax output. The model contains approximately 0.45 million parameters and was selected to ensure compatibility with low-power embedded devices while maintaining representative classification capability for comparative benchmarking experiments. This lightweight

structure enables evaluation of optimization techniques under realistic TinyML deployment conditions. The MobileNetV3 baseline architecture used for comparison contains approximately 5.4 million trainable parameters, enabling consistent benchmarking against the proposed compact TinyML architecture.

3.3 | Problem Statement in Embedded AI Deployment

One of the biggest challenges of embedded AI is the huge memory and computational needs of deep models, which are hard to implement on low-power microcontrollers with weak computing power. Classical convolutional neural networks are computationally demanding for deployment on low-power embedded platforms, which limits their suitability for real-time inference on resource-constrained edge devices.

3.4 | Lightweight Model Optimization Strategy

Model optimization techniques such as pruning and quantization, together with lightweight neural network architectures such as MobileNetV3 and TinyML, address these computational and memory limitations in embedded deployment environments. They reduce computation and model size without compromising accuracy. AI-specialized hardware accelerators such as TPUs and MCUs can also significantly enhance inference speed and efficiency and enable embedded AI in real-time systems.

3.5 | Implementation Details and Experimental Setup

The optimized models were implemented using TensorFlow Lite and TensorRT frameworks and deployed across heterogeneous embedded platforms including an ARM-based CPU (Raspberry Pi 4 Model B, 4 GB RAM), an NVIDIA Jetson Nano GPU platform (128-core Maxwell architecture), a Google Coral Edge TPU accelerator, and an STM32 microcontroller unit supporting TensorFlow Lite Micro. Model optimization and deployment were performed using TensorFlow Lite version 2.12, CUDA-enabled TensorRT acceleration for GPU inference, and TensorFlow Lite Micro for MCU execution to ensure hardware-aware benchmarking consistency.

Experiments were conducted using a subset of the ImageNet classification dataset consisting of 10 object categories with approximately 1000 images per category. The selected categories include representative everyday objects such as vehicles, animals, tools, and household items to simulate realistic embedded vision inference workloads under constrained deployment conditions. The dataset was divided into training and testing subsets using an 80:20 split. All images were resized to 224×224 pixels and normalized prior to inference to maintain uniform preprocessing across platforms.

Model training was performed offline on a workstation environment using the Adam optimizer with a learning rate of 0.001 and batch size of 32 for 25 epochs. Cross-entropy loss was used as the objective function, and data augmentation included random

horizontal flipping and normalization to improve generalization performance. A dropout rate of 0.3 was applied to the fully connected layer to reduce overfitting during training.

Only inference benchmarking was conducted on embedded hardware platforms. Performance profiling tools were used to measure inference latency, power consumption, memory footprint, and classification accuracy under identical runtime conditions. All latency and power measurements were averaged over repeated inference runs to ensure reliable cross-platform comparison between baseline and optimized models. All experiments were conducted under identical preprocessing procedures, runtime configurations, and batch-size conditions to ensure fair benchmarking consistency across heterogeneous hardware platforms.

4 | Results and Discussion

Embedded AI system performance depends on the trade-off between classification accuracy, computational efficiency, power consumption, memory usage, and real-time responsiveness. This section presents and discusses the testing results of two minimalist AI models, MobileNetV3 and a custom TinyML model, on a range of embedded boards. The analysis encompasses model accuracy comparisons, memory usage, inference latency on different hardware accelerators, and the effectiveness of optimizations such as quantization and pruning. These results demonstrate the feasibility of deploying optimized lightweight AI models on resource-constrained edge devices with minimal performance degradation. All latency and power measurements were obtained as averages over five repeated inference runs to reduce variability associated with hardware execution conditions.

4.1 | Rationale for Model Selection

MobileNetV3 and TinyML were chosen for comparative evaluation since they represent two lightweight AI design philosophy extremes. MobileNetV3 is a lightweight convolutional neural network designed for efficient inference on mobile and embedded platforms using depthwise separable convolutions and neural architecture search. It is vision-centric and a proven benchmark for performance-focused lightweight models. Unlike that, though, the TinyML model employed here is a specially designed architecture optimized for ultra-low-resource environments where memory and power are limited. Its low footprint renders it extremely well suited for deployment on the microcontroller scale at the expense of some accuracy.

These two models portray a striking contrast of performance-optimized and resource-optimized implementations, enabling a global comprehension of the trade-offs in infusing AI.

4.2 | Evaluation Parameters and Methodology

The assessment was derived from four major indicators:

- *Accuracy*—Model accuracy was evaluated using classification performance on the selected image dataset

- *Model Size*—Every model's megabyte (MB) size should be provided to guarantee storage and memory compatibility with embedded devices.
- *Inference Latency*—Latency in milliseconds was also compared across various hardware platforms: Central Processing Unit (CPU), Graphics Processing Unit (GPU), Tensor Processing Unit (TPU), Microcontroller Unit (MCU).
- *Energy Efficiency*—Power was measured in watts (W) to account for the effect of model run time on power-limited platforms like IoT sensors or wearables.

This diversified analysis provides an overall impression of model performance in real embedded system deployment. Optimization experiments were performed with optimized inference runtimes like TensorFlow Lite, TensorRT, and vendor-provided SDKs for embedded accelerators. Hardware profiling tools and simulation toolchains were utilized for translation of high-level performance data.

4.3 | Interpreting the Trade-Offs

Embedded AI presents system designers and engineers with an intimidating set of trade-offs. Higher model accuracy may require larger models and more latency, and strong optimization methods can minimize resource consumption but at the expense of inference efficiency. One must realize the trade-offs during system design for AI-enabled systems within application-specific constraints

For example, for real-time health monitoring or autonomous robotic control, the efficiency of energy and latency could be more important than marginal accuracy improvement in the model. But in industrial defect detection use cases, where false negatives and false positives are prohibitively costly, accuracy can need to be critical even at the expense of increased computational load.

In this context, the results presented in the following subsections provide insights into:

- How different models scale in terms of performance and resource efficiency.
- The effectiveness of quantization and pruning in reducing latency and power consumption.
- The suitability of various hardware platforms for specific AI workloads in embedded environments.

By interpreting these results, we aim to guide design decisions for embedded AI deployments, especially in contexts where constraints are stringent and application requirements are non-negotiable. These observations confirm that model-level optimization strategies can be effectively aligned with hardware-specific execution characteristics to achieve balanced improvements in latency, memory footprint, and energy efficiency across heterogeneous deployment environments.

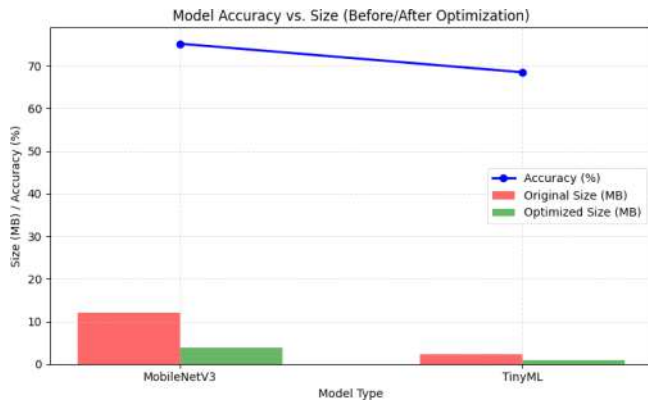


FIGURE 4 | Comparison of model accuracy and size.

4.4 | Structure of the Discussion

The discussion is structured around a series of performance tables and graphs. These include:

- A comparison of accuracy and model size between MobileNetV3 and the TinyML model.
- Latency measurements across CPU, GPU, TPU, and MCU platforms.
- Power consumption and latency improvements are achieved through quantization and pruning.

Each set of findings is examined on a practical level to help derive strong recommendations for the design of embedded AI systems. The research not only substantiates the feasibility of using cutting-edge AI at the edge but also underlines how careful model choice and optimization can create new possibilities for intelligent, distributed computing [39, 40].

Figure 4 compares the model size and classification accuracy of MobileNetV3 and the custom TinyML architecture before and after optimization. Prior to optimization, the MobileNetV3 model size was approximately 12.1 MB, whereas the TinyML model was significantly smaller at about 2.3 MB. After applying optimization techniques, the model sizes were substantially reduced to approximately 3.8 MB for MobileNetV3 and 0.9 MB for the TinyML model, demonstrating effective compression suitable for embedded deployment. In terms of classification performance, MobileNetV3 achieved an accuracy of 75.2%, while the TinyML model achieved 68.5%. Despite the reduction in model size, classification accuracy remained within acceptable variation ranges for resource-constrained embedded AI applications. These results indicate that optimization techniques can significantly reduce memory requirements while preserving reliable predictive performance for deployment on edge devices. A comparative summary of accuracy and model size is presented in Table 2 [34] and [35].

The Rationale for Model Selection describes a comparison of models such as MobileNetV3 against a bespoke TinyML design based on the performance (accuracy)-efficiency (size, latency, energy) trade-off.

TABLE 2 | Comparison of model accuracy and size for MobileNetV3 and custom TinyML architectures.

Models	Accuracy (%)	Size (MB)
MobileNetV3	75.2	12.1
TinyML (Custom)	68.5	2.3

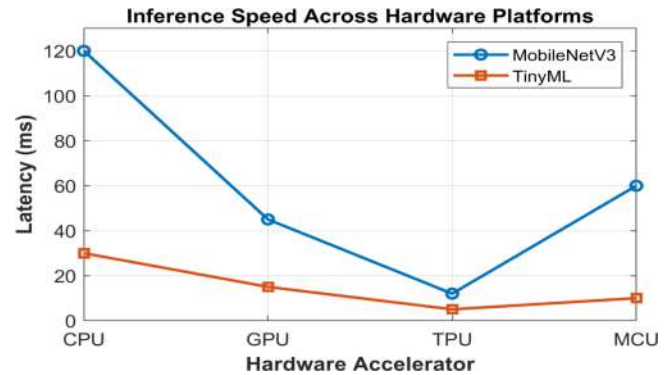


FIGURE 5 | Inference speed across hardware platform.

4.4.1 | Model Efficiency

Accuracy-to-model-size ratio is commonly used as a lightweight deployment suitability indicator because it reflects predictive capability relative to storage requirements on constrained embedded devices. A common metric to evaluate efficiency is the accuracy per model size as given in Equation (1) [41].

$$E = \frac{\text{Accuracy (\%)}}{\text{Model Size (MB)}} \quad (1)$$

Figure 5 shows the latency of inference, in milliseconds, of MobileNetV3 and TinyML models on different hardware accelerators: CPU, GPU, TPU, and MCU. MobileNetV3 shows significantly higher latency than TinyML on all the architectures since it is bigger and more complex in model size. On the CPU, MobileNetV3 is the most sluggish with the highest latency of approximately 120 ms, while TinyML is significantly faster with a latency of approximately 30 ms. The two models are faster on the GPU and TPU, where the TPU performs best—MobileNetV3 reduces latency to approximately 12 ms, while TinyML even performs faster with a response of about 5 ms. But when executed on an MCU, the latency of MobileNetV3 goes up to approximately 60 ms, while TinyML stays efficient with a latency of almost 10 ms. This comparison indicates that TinyML is more appropriate for highly resource-starved platforms such as micro-controllers, providing uniformly low latency, while MobileNetV3, while more capable, gains immensely from high-speed accelerators such as GPUs and TPUs. Inference Latency (ms) Across Hardware Platforms for MobileNetV3 and TinyML Models as depicted in Table 3 [36–38].

4.4.2 | Computational Cost of Convolutional Layers

4.4.2.1 | MobileNetV3 (Depth-Wise Separable Convolutions). As mentioned earlier, MobileNetV3 replaces

TABLE 3 | Inference latency (ms) across hardware platforms for MobileNetV3 and TinyML models.

Hardware	CPU (ms)	GPU (ms)	TPU (ms)	MCU (ms)
MobileNetV3	120	45	12	60
TinyML	30	15	5	10

standard convolutions with depth-wise separable convolutions, significantly reducing the number of FLOPs, Where FLOP stands for Floating Point Operation.

FLOPs formula for Depth Separable Convolution [42] as given in Equation (2)

$$\text{FLOP}_{\text{MobileNetV}_3} = D_k^2 \cdot M \cdot D_f^2 + M \cdot N \cdot D_f^2 \quad (2)$$

where D_k = Kernel size; M = input channel; N = output channel; D_f = spatial dimension of the feature map.

4.4.2.2 | TinyML Models (Custom, Ultra-Efficient). TinyML models are not standardized like MobileNetV3, usually custom-designed for extremely low-resource devices (e.g., Cortex-M MCUs with < 256 KB RAM).

If a TinyML model uses very small convolutional layers (e.g., 1 or 2 layers), the total FLOPs can be approximated as Equation (3) [43].

$$\text{FLOP}_{\text{TinyML}} = \sum_{i=1}^L 2 \cdot D_k^2 \cdot M_i \cdot N_i \cdot D_f^2 \quad (3)$$

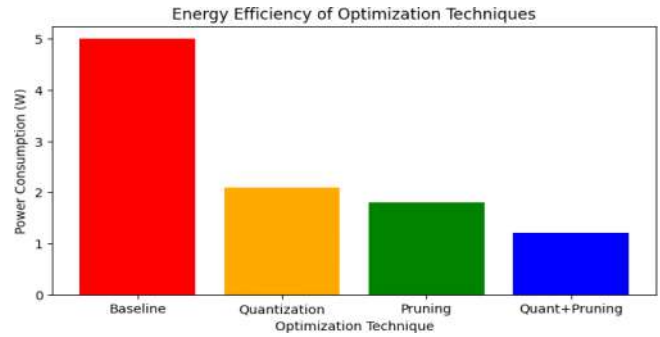
Figure 6 illustrates how some of the various optimizations influence the power consumption, the metric of energy efficiency, on energy. Without optimization, there is the greatest level of power consumption at about 5 W by the Baseline model. Quantization reduces the level of power required to about 2.1 W, illustrating gains from decreasing the level of precision numerically. Pruning, or cutting off unnecessary model parameters, decreases the amount of power consumption to about 1.8 W. The most power-efficient outcome is obtained by pairing both methods—Quantization and Pruning—and this lowers power consumption to almost 1.2 W. This trend indicates that optimization techniques not only minimize model size and computation time but also significantly improve energy efficiency, making them incredibly useful for deploying machine learning models on power-limited devices. The Effect of Optimization Methods on Power Usage and Latency in Embedded AI Models is illustrated in Table 4 [44–46].

4.4.3 | Latency-Energy Product (LEP)

To jointly consider real-time response and energy consumption [47], we define the Latency-Energy Product as given in Equation (4).

$$\text{LEP} = L * P \quad (4)$$

where L = latency (ms), P = power consumption (W).

**FIGURE 6** | The energy efficiency of optimization techniques.**TABLE 4** | Impact of optimization techniques on power consumption and latency in embedded AI models.

Techniques	Power (W)	Latency (ms)
Baseline	5.0	100
Quantization	2.1	45
Pruning	1.8	38
Quant + pruning	1.2	25

Power consumption values were obtained using onboard hardware monitoring utilities and averaged across repeated inference runs under identical execution conditions.

This study differs from previous embedded AI benchmarking studies in presenting a combined perspective of Embedded AI through the twin lens of hardware and software integration, particularly emphasizing that of real-time edge deployment. In contrast to other studies that address embedded AI from a broad perspective, this study targets the relevance of embedded GPUs such as NVIDIA Jetson and advanced acceleration techniques such as CUDA programming, tensor cores, and mixed-precision computing. Apart from that, it also introduces a structured benchmarking workflow that contrasts light models (MobileNetV3 and one author's own custom TinyML architecture) over multiple hardware targets—CPU, GPU, TPU, and MCU—concerning the performance metrics including accuracy, model size, inference latency, and power consumption. Through this comparison to real life, the paper highlights performance vs. resource trade-offs and justifies the relevance of using AI in resource-constrained environments. The utilization of systematic optimization techniques such as quantization and pruning also justifies the contribution of the work toward enhancing energy efficiency and computational feasibility in real-life embedded systems.

4.4.4 | Overall Performance Comparison

Optimization results reported in Table 5 correspond to MobileNetV3 after sequential application of quantization and pruning techniques unless otherwise specified. To provide a holistic understanding of the trade-offs between accuracy, memory footprint, latency, and energy consumption, a consolidated comparison of all evaluated models and optimization techniques is presented in Table 5. This unified view helps system designers quickly identify the most suitable model configuration for

TABLE 5 | Comprehensive comparison of embedded AI models and optimization.

Model/technique	Accuracy (%)	Size (MB)	CPU latency (ms)	MCU latency (ms)	Power (W)	Energy efficiency level	Best use case
MobileNetV3 (Baseline)	75.2	12.1	120	60	5	Low efficiency	High-accuracy edge inference on GPU/TPU platforms
TinyML (Baseline)	68.5	2.3	30	10	5	Moderate efficiency	Ultra-low-resource embedded devices
Quantized model	74–75	3.8	45	25	2.1	Improved efficiency	IoT systems requiring reduced power and latency
Pruned model	73–74	3.2	38	22	1.8	High efficiency	Battery-powered embedded platforms
Quantization + pruning	72–74	0.9–1.5	25	12	1.2	Very high efficiency	Ultra-low-power MCU and wearable deployments

TABLE 6 | Throughput and memory utilization of lightweight models.

Model	Throughput (FPS)	RAM usage (MB)
MobileNetV3	20	180
TinyML	65	40

specific embedded AI applications [48], particularly under strict power and resource constraints by using Equation (5). This efficiency metric integrates prediction accuracy with computational latency and energy consumption to provide a unified indicator of deployment suitability for resource-constrained embedded platforms.

$$E = \frac{\text{Accuracy}}{\text{Latency} * \text{Power}} \quad (5)$$

4.4.5 | Additional Performance Metrics

In addition to the primary benchmarking metrics, two supplementary deployment-level performance indicators, throughput and runtime memory utilization, are presented in Table 6 to further evaluate inference efficiency on embedded platforms. Throughput (frames per second) quantifies the number of inferences processed per second and reflects the real-time processing capability of the deployment system. Runtime memory utilization measures RAM consumption during inference execution and indicates the feasibility of deploying optimized models on resource-constrained devices. These additional indicators further support the suitability of lightweight neural network configurations for embedded AI environments.

Throughput values were calculated based on sustained inference execution over multiple input batches to reflect realistic embedded deployment performance conditions. Higher throughput values correspond to reduced per-frame inference latency under identical batch-size and runtime configurations across platforms and therefore indicate improved suitability for real-time embedded deployment scenarios. Runtime memory usage was measured using platform-specific profiling utilities during inference execution after model loading, ensuring consistent comparison across heterogeneous hardware targets.

5 | Conclusion

This study presents a cross-platform benchmarking framework that systematically evaluates lightweight neural network optimization strategies across heterogeneous embedded hardware architectures using unified performance metrics. Unlike prior studies that evaluate lightweight neural networks on individual hardware platforms, the proposed framework provides a unified evaluation across CPU, GPU, TPU, and MCU architectures under consistent experimental conditions, enabling systematic comparison of optimization strategies for resource-constrained embedded AI deployment. The integration of artificial intelligence within embedded systems, referred to as Embedded AI, represents an important step toward decentralized intelligent computation on resource-constrained hardware platforms. Offering on-device inference and decision-making, Embedded AI sidesteps reliance on cloud infrastructure, thereby reducing latency, enhancing privacy, and optimizing energy efficiency. This paper has analyzed the architectural, algorithmic, and hardware-level advances that enable such integration, even in highly resource-constrained environments. When testing light models like MobileNetV3 and hand-crafted TinyML architectures, there is explicit evidence that successful trade-offs between model accuracy, size, and energy consumption are essential in the pursuit of optimizing for real-world embedded AI. Optimizations like pruning and quantization were applied to significantly constrain memory and latency with only minor reductions in classification accuracy within acceptable deployment limits for models, leaving them ideally placed to execute on microcontrollers and other edge devices.

Additionally, the role of specialized hardware accelerators such as GPUs, TPUs, and AI-centric MCUs has been seen to be substantial in accelerating AI workloads at the edge. Benchmark results indicated the way the performance can be dramatically platform-dependent, with TinyML models outperforming larger networks in ultra-low-latency, low-power use cases. The results also highlight the importance of model selection and optimization pipeline based on application-specific requirements. Applications including medical monitoring, industrial automation, smart home control, and Internet of Things systems demonstrate the practical impact of embedded AI in enabling real-time intelligent decision-making at the edge.

The ability to perform real-time analytics and autonomous control at the local level transforms how systems interact with the world around them. That being noted, issues of heterogeneity in hardware, portability of models, and limiting real-time performance are topics that will be researched. Next-generation breakthroughs in the future in areas like ultra-efficient AI architecture, software and hardware co-design for optimization, and paradigms like neuromorphic computing and federated learning can be expected to take this space further. With more democratized AI capabilities becoming accessible through embedded platforms, they will enable a new era of smart, adaptive, and secure systems working together in harmony at the edge.

5.1 | Limitations and Future Work

Despite the promising results, this study has certain limitations. The experimental evaluation is restricted to a limited set of lightweight models, namely MobileNetV3 and a custom TinyML architecture, and may not fully represent the performance of other emerging neural networks. Benchmarking is conducted on selected embedded platforms, and results may vary across different hardware configurations and real-world deployment conditions. The evaluation is limited to a reduced ImageNet subset and may not fully capture performance variations across domain-specific embedded vision datasets. Additionally, the experiments primarily focus on inference performance, while training efficiency and large-scale field testing are not considered. Future work will extend this study by incorporating additional lightweight architectures, diverse datasets, and broader hardware platforms, along with real-time deployment in practical applications to further validate scalability and robustness. The proposed benchmarking framework can support system designers in selecting appropriate lightweight neural network configurations for real-time edge intelligence applications where computational resources and energy availability are limited.

Author Contributions

Vidapankal Mohammad Fridous: conceptualization, investigation, writing – original draft, writing – review and editing, formal analysis, data curation, resources, software. **Abhishek Agarwal:** conceptualization, funding acquisition, writing – original draft, visualization, writing – review and editing, methodology, formal analysis, project administration, supervision, resources, data curation, investigation. **K. B. Bhaskar:** conceptualization, writing – original draft, writing – review and editing, validation, visualization, software, formal analysis, data curation, resources, project administration, supervision, investigation. **V. Nandagopal:** conceptualization, investigation, data curation, formal analysis, validation, writing – review and editing, writing – original draft. **N. Sivakamasundari:** conceptualization, investigation, writing – original draft, validation, visualization, formal analysis, data curation, resources, project administration, software.

Acknowledgments

The authors would like to acknowledge the institutional support provided for carrying out this research work. No external funding agency was involved in the design, implementation, analysis, or reporting of this study.

Funding

The authors have nothing to report.

Ethics Statement

The authors have nothing to report.

Consent

All authors have reviewed the manuscript and consent to its submission for publication.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The dataset used in this study was derived from a publicly available ImageNet subset. Additional processed data supporting the findings of this study are available from the corresponding author upon reasonable request.

Peer Review

For transparency, the peer review documents associated with this article are available at <https://doi.org/10.1002/eng2.70814>.

References

1. F. Oliveira, D. G. Costa, F. Assis, and I. Silva, "Internet of Intelligent Things: A Convergence of Embedded Systems, Edge Computing and Machine Learning," *Internet of Things* 26 (2024): 101153.
2. X. Huang, H. Wang, S. Qin, and S.-K. Tang, "Embedded Artificial Intelligence: A Comprehensive Literature Review," *Electronics (Basel)* 14, no. 17 (2025): 3468.
3. R. Singh and S. S. Gill, "Edge AI: A Survey," *Internet of Things and Cyber-Physical Systems* 3 (2023): 71–92.
4. C.-E. Vatile, A.-A. Ulmămei, and C. Biră, "Image Processing Hardware Acceleration—A Review of Operations Involved and Current Hardware Approaches," *Journal of Imaging* 10, no. 12 (2024): 298.
5. C. Silvano, D. Ielmini, F. Ferrandi, et al., "A Survey on Deep Learning Hardware Accelerators for Heterogeneous HPC Platforms," *ACM Computing Surveys* 57, no. 11 (2025): 1–39.
6. P. Ruiz-Barroso, F. M. Castro, and N. Guil, "Real-Time Unsupervised Video Object Detection on the Edge," *Future Generation Computer Systems* 167 (2025): 107737.
7. T. Abdelzaher, Y. Hu, D. Kara, et al., "The Bottlenecks of AI: Challenges for Embedded and Real-Time Research in a Data-Centric Age," *Real-Time Systems* 61, no. 2 (2025): 185–236.
8. M. Shobanke, M. Bhatt, and E. Shittu, "Advancements and Future Outlook of Artificial Intelligence in Energy and Climate Change Modeling," *Advances in Applied Energy* 17 (2025): 100211.
9. C. M. Bhushan, P. Koppuravuri, N. Prasanthi, et al., "Deploying TinyML for Energy-Efficient Object Detection and Communication in Low-Power Edge AI Systems," *Scientific Reports* 15, no. 1 (2025): 44299.
10. A. Jain and S. Bhullar, "AI-Driven Wearable Health Devices With Health-Aware Control and Secure Prognostics: Experimental and Simulation-Based Validation," *Array* 28 (2025): 100532.
11. A. D. Scaife, "Improve Predictive Maintenance Through the Application of Artificial Intelligence: A Systematic Review," *Results in Engineering* 21 (2024): 101645.

12. J. S. Fata, W. M. Elmannai, and K. Elleithy, "Balancing Performance and Cost—FPGA-Based CNN Accelerators for Edge Computing: Status Quo, Key Challenges, and Prospective Innovations," *IEEE Access* 13 (2025): 142852–142877.
13. P. Gao and M. Adnan, "Overview of Emerging Electronics Technologies for Artificial Intelligence: A Review," *Materials Today Electronics* 11 (2025): 100136.
14. K. P. Seng, P. J. Lee, and L. M. Ang, "Embedded Intelligence on FPGA: Survey, Applications and Challenges," *Electronics (Basel)* 10, no. 8 (2021): 895.
15. H.-Y. Lin, "Embedded Artificial Intelligence: Intelligence on Devices," *Computer (Long Beach, Calif)* 56, no. 9 (2023): 90–93.
16. P. Dini, L. Diana, A. Elhanashi, and S. Saponara, "Overview of AI-Models and Tools in Embedded IIoT Applications," *Electronics (Basel)* 13, no. 12 (2024): 2322.
17. M. S. Diab and E. Rodriguez-Villegas, "Embedded Machine Learning Using Microcontrollers in Wearable and Ambulatory Systems for Health and Care Applications: A Review," *IEEE Access* 10 (2022): 98450–98474.
18. K. Sun, X. Wang, X. Miao, and Q. Zhao, "A Review of AI Edge Devices and Lightweight CNN and LLM Deployment," *Neurocomputing* 614 (2025): 128791.
19. E. Prasetyo, R. Purbaningtyas, R. D. Adityo, N. Suciati, and C. Fatichah, "Combining MobileNetV1 and Depthwise Separable Convolution Bottleneck With Expansion for Classifying the Freshness of Fish Eyes," *Information Processing in Agriculture* 9, no. 4 (2022): 485–496.
20. A. G. Howard, M. Zhu, B. Chen, et al., (2017), "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications".
21. C. Lin, P. Yang, Q. Wang, Z. Qiu, W. Lv, and Z. Wang, "Efficient and Accurate Compound Scaling for Convolutional Neural Networks," *Neural Networks* 167 (2023): 787–797.
22. A. R. Altaf, A. Altaf, and F. U. Rehman, "Modified EfficientNet-B0 Architecture Optimized With Quantum-Behaved Algorithm for Skin Cancer Lesion Assessment," *Diagnostics* 15, no. 24 (2025): 3245.
23. P. V. Dantas, W. Sabino da Silva, L. C. Cordeiro, and C. B. Carvalho, "A Comprehensive Review of Model Compression Techniques in Machine Learning," *Applied Intelligence* 54, no. 22 (2024): 11804–11844.
24. S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks With Pruning, Trained Quantization and Huffman Coding," *International Conference on Learning Representations (ICLR)* (2016), <https://doi.org/10.48550/arXiv.1510.00149>.
25. A. Setyanto, T. Bayu Sasongko, M. A. Fikri, et al., "Knowledge Distillation in Object Detection for Resource-Constrained Edge Computing," *IEEE Access* 13 (2025): 18200–18214.
26. A. Moslemi, A. Briskina, Z. Dang, and J. Li, "A Survey on Knowledge Distillation: Recent Advancements," *Machine Learning With Applications* 18 (2024): 100605.
27. G. Hinton, O. Vinyals, and J. Dean (2015), "Distilling the Knowledge in a Neural Network," arXiv:1503.02531. Presented at the NIPS Deep Learning Workshop.
28. A. Samanta, I. Hatai, and A. K. Mal, "A Survey on Hardware Accelerator Design of Deep Learning for Edge Devices," *Wireless Personal Communications* 137, no. 3 (2024): 1715–1760.
29. S. Alam, C. Yakopcic, Q. Wu, M. Barnell, S. Khan, and T. M. Taha, "Survey of Deep Learning Accelerators for Edge and Emerging Computing," *Electronics (Basel)* 13, no. 15 (2024): 2988.
30. W. Roth, G. Schindler, B. Klein, et al., "Resource-Efficient Neural Networks for Embedded Systems," *Journal of Machine Learning Research* 25, no. 50 (2024): 1–51.
31. Y. Chen, C. Hawkins, K. Zhang, Z. Zhang, and C. Hao, "3U-EdgeAI: Ultra-Low Memory Training, Ultra-Low Bitwidth Quantization, and Ultra-Low Latency Acceleration," *Proceedings of the 2021 Great Lakes Symposium on VLSI* (2021): 157–162, <https://doi.org/10.1145/3453688.3461738>.
32. P. P. Ray, "A Review on TinyML: State-Of-The-Art and Prospects," *Journal of King Saud University, Computer and Information Sciences* 34, no. 4 (2022): 1595–1623.
33. N. Schizas, A. Karras, C. Karras, and S. Sioutas, "TinyML for Ultra-Low Power AI and Large Scale IoT Deployments: A Systematic Review," *Future Internet* 14, no. 12 (2022): 363.
34. A. Howard, M. Sandler, B. Chen, et al., "Searching for MobileNetV3," *IEEE/CVF International Conference on Computer Vision (ICCV)* 2019 (2019): 1314–1324.
35. C. Banbury, V. J. Reddi, P. Torelli, et al., "MLPerf Tiny Benchmark," *35th Conference on Neural Information Processing Systems (NeurIPS 2021)* (2021): 2106.07597, <https://doi.org/10.48550/arXiv.2106.07597>.
36. N. P. Jouppi, D. Hyun Yoon, M. Ashcraft, et al., "Ten Lessons From Three Generations Shaped Google's TPUPv4i: Industrial Product," *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)* (2021): 1–14, <https://doi.org/10.1109/ISCA52012.2021.00010>.
37. R. David, J. Duke, A. Jain, et al., "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems," *Proceedings of Machine Learning and Systems* (2021): 2010.08678, <https://doi.org/10.48550/arXiv.2010.08678>.
38. A. Ignatov, R. Timofte, A. Kulik, et al., "AI Benchmark: All About Deep Learning on Smartphones in 2019," *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops* (2019): 3617–3635.
39. D. Saha, M. P. Mangukia, and A. Manickavasagan, "Real-Time Deployment of MobileNetV3 Model in Edge Computing Devices Using RGB Color Images for Varietal Classification of Chickpea," *Applied Sciences* 13, no. 13 (2023): 7804.
40. M. Giordano, N. Baumann, M. Crabolu, R. Fischer, G. Bellusci, and M. Magno, "Design and Performance Evaluation of an Ultralow-Power Smart IoT Device With Embedded TinyML for Asset Activity Monitoring," *IEEE Transactions on Instrumentation and Measurement* 71 (2022): 1–11.
41. O. Rainio, J. Teuho, and R. Klén, "Evaluation Metrics and Statistical Tests for Machine Learning," *Scientific Reports* 14, no. 1 (2024): 6086.
42. L. Dang, P. Pang, and J. Lee, "Depth-Wise Separable Convolution Neural Network With Residual Connection for Hyperspectral Image Classification," *Remote Sensing* 12, no. 20 (2020): 3408.
43. N. N. Alajlan and D. M. Ibrahim, "TinyML: Enabling of Inference Deep Learning Models on Ultra-Low-Power IoT Edge Devices for AI Applications," *Micromachines (Basel)* 13, no. 6 (2022): 851.
44. W. Zhao, Y. Bai, Q. Sun, et al., "A High-Performance Accelerator for Super-Resolution Processing on Embedded GPU," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, no. 10 (2023): 3210–3223.
45. J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny Deep Learning on IoT Devices," *Proceedings 34th Conference on Neural Information Processing Systems (NeurIPS)* (2020), <https://doi.org/10.48550/arXiv.2007.10319>.
46. T.-J. Yang, A. Howard, B. Chen, et al., "NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications," In *Computer Vision – ECCV 2018, LNCS 11214* (Springer, 2018), 289–304.

47. B. Wang, A. Kozhokanova, C. Terboven, and M. Mueller, "RLP: Power Management Based on a Latency-Aware Roofline Model," *IEEE International Parallel and Distributed Processing Symposium (IPDPS)* 2023 (2023): 446–456.
48. J. Cao, Z. Yu, B. Zhu, M. Cao, and J. Yang, "Construction and Efficiency Analysis of an Embedded System-Based Verification Platform for Edge Computing," *Scientific Reports* 15, no. 1 (2025): 26114.